Academic Scheduling and Exam Collision Management System

Project Description, Work Packages, and 4-Week Internship Roadmap

Project type	**Single team project for 3 intern students**
Duration	4 weeks
Primary users	Faculty/programme admins and department sub-accounts
Main objective	Build a web-based application for weekly course scheduling and midterm/final/makeup exam planning with collision detection

This document defines the target MVP (Minimum Viable Product), expected features, team split, implementation work packages, acceptance criteria, and week-by-week roadmap. Students should treat this as the baseline scope and clarify unresolved rules during the first project meeting.

1. Project Overview

Project goal. Develop a web-based scheduling application that helps academic units manage weekly course timetables and exam schedules while detecting conflicts before schedules are finalized.

Context. Each faculty or programme will have an admin user. The admin creates one or more scheduling workgroups, defines departments, invites department-level sub-accounts, and manages shared classrooms. Department sub-accounts maintain their own course lists and participate in scheduling. All users register using university email addresses.

Core value. The system should reduce manual spreadsheet work, prevent classroom/lecturer/cohort conflicts, and provide clear reports for course and exam timetables.

1.1 Problem Statement

- Academic course and exam scheduling is often handled manually in spreadsheets, which makes errors difficult to detect.
- Classroom capacity, lecturer availability, and department/year/semester overlaps are hard to track across multiple departments.
- Midterm, final, and makeup exam schedules require one-off date/time planning, which uses different logic from weekly course scheduling.
- Multiple departmental users may need to collaborate, so authorization and ownership rules must be explicit.

1.2 Expected MVP Outcome

- A working web application with authentication, admin and department sub-account roles, and invitation-based account completion.
- CRUD (Create, Read, Update, Delete) screens for departments, users, courses, classrooms, weekly course sessions, and exam sessions.
- A collision detection engine that checks weekly schedules and exam schedules before saving or publishing.
- Filterable timetable views by department, classroom, lecturer, and exam type.
- Basic reporting/export support, at minimum CSV or Excel export for course and exam schedules.
- Documentation, test data, deployment instructions, and a final demo.

1.3 Out of Scope for the 4-Week MVP

- Full automatic timetable optimization. The MVP should detect and explain conflicts; it does not need to solve all scheduling automatically.
- Integration with university LDAP/SSO or official student information systems, unless mock integration is added as an optional extension.
- Advanced proctor assignment, lecturer preference collection, or student-level enrollment conflict detection beyond cohort-level rules.
- Mobile-native applications. Responsive web design is enough.
- Production-grade high availability, although the app should be deployable and secure enough for demonstration.

2. Users, Roles, and Permissions

The application should implement role-based access control. For the MVP, the following roles are sufficient.

Role	Responsibilities
Faculty/Programme Admin	Owns a scheduling group; creates departments and sub-accounts; manages all courses, classrooms, weekly schedules, exams, and reports within the group.
Department Sub-account	Assigned to one or more departments; manages the course list and schedules for assigned departments; can add/remove classrooms for the group if this permission is enabled.
System Seed/Admin User	Optional development-only role used to create the first faculty/programme admin and reset demo data.

2.1 Permission Matrix

Action	Admin	Department Sub-account

Create workgroup	Yes	No
Create/edit departments	Yes	No
Invite sub-accounts	Yes	No
Assign users to departments	Yes	No
Manage assigned department courses	Yes	Yes, assigned departments only
Manage classrooms	Yes	Yes, if permission is enabled
Create weekly course sessions	Yes	Yes, assigned departments only
Create midterm/final/makeup exams	Yes	Yes, assigned departments only
View all schedules in workgroup	Yes	Recommended: read-only
Override hard collisions	Optional	No

2.2 Account Invitation Flow

1. Admin enters the user name, university email address, role, and assigned department(s).
2. System validates that the email belongs to an allowed university domain. The domain should be configurable; do not hard-code a single domain.
3. System creates a pending user record and a secure invitation token with an expiration date.
4. System sends an email containing a one-time account-completion link.
5. User opens the link, confirms the email, sets a password, and activates the account.
6. Expired or already-used invitation links must not work. Admin should be able to resend an invitation.

> **MVP email delivery may use a real SMTP service, a sandbox mail catcher, or console-based mock email output. The code structure should make real SMTP configuration possible through environment variables.**

3. Functional Requirements

3.1 Workgroups and Departments

- Admins can create and edit a workgroup representing a faculty, programme, or scheduling unit.
- Admins can add departments under the workgroup.
- Admins can assign multiple sub-accounts to one department and one sub-account to multiple departments.
- All data must be isolated by workgroup so that users cannot access another faculty/programme schedule.

3.2 Course Management

- Sub-accounts can create, edit, list, search, and deactivate/remove courses for their assigned department(s).
- Each course must store department, year, semester, course code, section number, course name, lecturer, and classroom information.
- The course code + section number + department + semester combination should be unique within a workgroup.
- Lecturer may be a text field in the MVP; optional extension: create a lecturer entity with email and availability.
- Classroom information may be stored as a default/preferred classroom on the course and as actual assigned classroom(s) on schedule entries.

3.3 Classroom Management

- Admins and permitted sub-accounts can add, edit, deactivate, and remove classrooms for the workgroup.
- Each classroom must store building name, classroom ID/code, and maximum student capacity.
- Classroom ID should be unique within a building and workgroup.
- Optional fields: floor, equipment, exam suitability, accessibility notes, and active/passive status.

3.4 Weekly Course Schedule

- Users can place course sessions into weekly timetable slots from Monday to Friday, 08:30-17:30.
- The timetable consists of 45-minute teaching slots followed by 15-minute breaks.
- Slot starts are 08:30, 09:30, 10:30, 11:30, 12:30, 13:30, 14:30, 15:30, and 16:30.

- A course session can occupy one or more consecutive 45-minute teaching slots.
- The UI should provide a weekly grid view and a form-based fallback for adding/editing sessions.

3.5 Exam Scheduling

- Users can create midterm, final, and makeup exam sessions for courses.
- Each exam should store course, exam type, date, start time, duration, classroom(s), lecturer/responsible person, and notes.
- The system should support one-room exams in the MVP. Multi-room exams may be implemented as an optional extension using an exam-room relation.
- Exam schedules should be filterable by department, exam type, date range, classroom, year, semester, and lecturer.
- Users should be able to export exam schedules for publication or review.

3.6 Collision Detection and Conflict Reports

- The system must check collisions before creating or updating weekly course sessions and exam sessions.
- Conflicts should be reported with clear explanations, not only generic error messages.
- Hard conflicts should normally block saving. Warnings may allow saving with a visible note.
- The user should be able to run a full conflict scan for the entire workgroup and view a conflict report.

3.7 Dashboard and Reporting

- Dashboard should show counts for departments, courses, classrooms, scheduled weekly sessions, exams, and unresolved conflicts.
- Reports should include weekly course timetable, classroom occupation view, lecturer view, and exam schedule list.
- Minimum export: CSV. Preferred export: XLSX. Optional export: PDF.

4. Collision Detection Rules

Collision detection is the most important technical part of the project. Implement it as a reusable service/module, not as scattered UI logic.

4.1 Core Time-Overlap Logic

> **Two events overlap if they occur on the same date/day and their time intervals intersect: startA < endB and startB < endA. Adjacent intervals such as 09:15-09:30 break and 09:30-10:15 class do not overlap.**

- Weekly course sessions are compared using day-of-week and slot intervals.
- Exam sessions are compared using concrete calendar dates and start/end times.
- The collision service should return structured results: severity, conflict type, affected objects, and human-readable explanation.

4.2 Weekly Course Collision Rules

Rule	Severity	Explanation
Classroom conflict	Hard error	Two course sessions use the same classroom at overlapping weekly times.
Lecturer conflict	Hard error	The same lecturer is assigned to two overlapping course sessions.
Department/year/semester cohort conflict	Hard error or warning	Courses from the same department, year, and semester overlap. This prevents students in the same cohort from attending both.
Duplicate course-section session	Warning	The same course section is scheduled twice at the same or overlapping time.
Capacity issue	Warning	Course expected enrollment exceeds classroom capacity. Requires optional enrollment field or manual expected capacity.
Out-of-window slot	Hard error	Session is outside Monday-Friday, 08:30-17:30 or does not align with allowed slot starts.

4.3 Exam Collision Rules

Rule	Severity	Explanation
Exam classroom conflict	Hard error	Two exams use the same classroom during overlapping date/time intervals.
Course duplicate exam	Hard error	The same course has more than one exam of the same type unless explicitly allowed.
Lecturer/responsible person conflict	Hard error or warning	The same lecturer/responsible person is assigned to overlapping exams.
Cohort exam conflict	Hard error or warning	Two exams for the same department/year/semester cohort overlap.
Capacity issue	Warning	Exam expected participant count exceeds total assigned classroom capacity.
Exam vs weekly course conflict	Configurable	During midterm/final weeks this may not matter; during normal weeks it should be checked if exams use normal teaching hours.

4.4 Conflict Message Examples

- “Classroom conflict: CENG2001-1 and MATH1001-2 both use Engineering E-B08 on Monday 10:30-11:15.”
- “Lecturer conflict: Dr. A is assigned to two sessions on Wednesday 13:30-15:15.”
- “Cohort warning: Computer Engineering Year 2 Fall courses CENG2001 and CENG2030 overlap on Tuesday 08:30-09:15.”
- “Exam conflict: Final exams for CENG1004 and CENG2001 overlap for Computer Engineering Year 1 on 2026-06-15.”

5. Suggested Data Model

The exact implementation may differ, but students should define a normalized relational model or equivalent document schema before coding. The following model is recommended for the MVP.

Entity	Important Fields	Notes / Constraints
User	id, name, email, password_hash, role, status, created_at	Email unique; must match allowed university domain; status: pending/active/disabled.
Workgroup	id, name, type, allowed_email_domain, created_by	Represents faculty/programme scheduling context.
Department	id, workgroup_id, name, code	Department code unique within workgroup.
DepartmentMembership	user_id, department_id, permission_level	Many-to-many relation between users and departments.
InvitationToken	id, user_id, token_hash, expires_at, used_at	Never store raw token if avoidable; store hash.
Classroom	id, workgroup_id, building, room_code, capacity, active	Unique: workgroup + building + room_code.
Course	id, department_id, year, semester, code, section_no, name, lecturer, expected_students	Unique: department + year + semester + code + section_no.
WeeklyScheduleEntry	id, course_id, classroom_id, day_of_week, start_slot, slot_count	Computed start/end times from slot table.
Exam	id, course_id, exam_type, date, start_time, duration_minutes, lecturer, notes	exam_type: MIDTERM, FINAL, MAKEUP.
ExamRoom	exam_id, classroom_id, capacity_allocation	Optional for MVP; useful for multi-room exams.
ConflictLog	id, workgroup_id, severity, type, message, entity_refs, resolved	Optional persistent log; conflict service can also compute live.

5.1 Slot Table

Slot	Start	End
1	08:30	09:15
2	09:30	10:15
3	10:30	11:15
4	11:30	12:15
5	12:30	13:15
6	13:30	14:15
7	14:30	15:15
8	15:30	16:15
9	16:30	17:15

A two-slot session starting at slot 3 runs from 10:30 to 12:15, with a 15-minute break between the two teaching slots. For collision detection, it occupies slots 3 and 4.

6. Suggested Screens and Architecture

6.1 Minimum Screens

- Login, account completion, forgot password or password reset placeholder.
- Admin dashboard with workgroup summary and conflict counts.
- Department management and user invitation/assignment pages.
- Course list page with filters and create/edit form.
- Classroom list page with filters and create/edit form.
- Weekly timetable grid with add/edit/delete course session actions.
- Exam schedule page with midterm/final/makeup filters and create/edit form.
- Conflict report page showing unresolved hard errors and warnings.
- Export page or export buttons on timetable/exam views.

6.2 Recommended Architecture

- **Frontend**
  - Use a component-based web UI such as React, Vue, Angular, or a server-rendered framework.
  - Weekly grid should be readable and filterable; drag-and-drop is optional, not required for MVP.
  - Forms should perform client-side validation but must still rely on backend validation.
- **Backend/API**
  - Use a structured backend framework such as Django, FastAPI, Spring Boot, Laravel, Express/NestJS, or equivalent.
  - Implement role-based authorization on every endpoint.
  - Place collision detection in a service layer with unit tests.
- **Database**
  - Use PostgreSQL, MySQL/MariaDB, or SQLite for early development with clear migration scripts.
  - Use foreign keys, uniqueness constraints, and soft delete or active/passive status where appropriate.
- **Email and configuration**
  - Configure SMTP and university domain rules through environment variables.
  - Use a mock/sandbox email mode for local development.

6.3 Security and Data Integrity Requirements

- Passwords must be hashed with a modern password hashing algorithm; never store plaintext passwords.
- Invitation and password reset tokens must expire and must be single-use.
- Users must not access another workgroup's data by changing URL IDs.
- All create/update/delete actions should be logged at least minimally with user and timestamp.
- Validate all user input on the backend.
- Do not hard-code secrets, SMTP credentials, or database passwords in source code.

7. Work Packages

The project should be organized as work packages so that three interns can work in parallel while integrating frequently.

WP	Title	Primary Owner	Main Output

WP0	Project setup and requirements	All	Repository, issue board, schema draft, wireframes, seed data plan.
WP1	Authentication, roles, invitations	Intern A	Login, admin/sub-account roles, university email validation, invitation flow.
WP2	Core data CRUD	Intern A + B	Workgroups, departments, users, courses, classrooms, and permissions.
WP3	Weekly scheduling module	Intern B + C	Weekly grid, slot model, create/edit/delete sessions, filters.
WP4	Exam scheduling module	Intern B + C	Midterm/final/makeup exam forms, date/time scheduling, exam views.
WP5	Collision detection engine	Intern C	Reusable conflict service for weekly courses and exams with unit tests.
WP6	Reports, exports, dashboard	Intern B	Timetable/exam reports, conflict dashboard, CSV/XLSX export.
WP7	Testing, deployment, documentation	All	Test cases, bug fixes, demo deployment, README, final presentation.

7.1 Detailed Package Descriptions

WP0 - Project Setup and Requirements

- Create Git repository, branch strategy, issue board, and coding conventions.
- Clarify ambiguous rules with the supervisor and record decisions in a project log.
- Prepare initial database schema and low-fidelity wireframes.
- Define seed data: example faculty, departments, classrooms, courses, lecturers, weekly sessions, and exams.
- Definition of done: repository runs locally, backlog is created, database design is reviewed, and students can demo initial wireframes.

WP1 - Authentication, Roles, and Invitations

- Implement login/logout/session or JWT-based authentication.
- Implement admin and department sub-account authorization.
- Implement university email domain validation.
- Implement invitation token creation, email sending/mock email, account completion, and token expiry.
- Definition of done: admin can invite a sub-account, user completes account setup, and unauthorized access is blocked.

WP2 - Core Data CRUD

- Implement CRUD for workgroups, departments, users, memberships, courses, and classrooms.
- Add search, filtering, and validation for course and classroom records.
- Enforce uniqueness and ownership constraints in the database.
- Definition of done: a realistic set of departments, courses, and classrooms can be entered and edited without database inconsistency.

WP3 - Weekly Scheduling Module

- Implement slot table and weekly timetable model.
- Build weekly grid view and create/edit/delete forms for schedule entries.
- Add filters by department, year, semester, classroom, and lecturer.
- Call collision service before saving schedule entries.
- Definition of done: users can build a weekly timetable and receive clear conflict messages.

WP4 - Exam Scheduling Module

- Implement exam entity and forms for midterm, final, and makeup exams.
- Support date, start time, duration, classroom, course, lecturer/responsible person, and notes.
- Build exam list/calendar view with filters.
- Call collision service before saving exam sessions.
- Definition of done: users can create exam schedules and detect exam-specific conflicts.

WP5 - Collision Detection Engine

- Implement generic interval-overlap logic.
- Implement weekly rules: classroom, lecturer, cohort, duplicate course-section, out-of-window slots, and capacity warnings.
- Implement exam rules: classroom, duplicate exam, lecturer/responsible person, cohort, capacity, and configurable exam-vs-course check.
- Return structured conflict objects with severity and explanation.
- Definition of done: unit tests cover positive, negative, edge, and boundary cases.

WP6 - Reports, Exports, and Dashboard

- Implement dashboard counts and unresolved conflict summary.
- Implement weekly timetable report by department, classroom, and lecturer.
- Implement exam schedule report by exam type/date/department.
- Implement CSV export; XLSX export is preferred if time allows.
- Definition of done: a supervisor can export a readable schedule from demo data.

WP7 - Testing, Deployment, and Documentation

- Write README with setup, environment variables, test users, and demo steps.
- Prepare test cases for permission checks, CRUD validation, collision detection, and exports.
- Deploy the application locally or to a staging environment.
- Prepare final presentation and demo script.
- Definition of done: the application can be installed from README and demonstrated end-to-end.

8. Suggested Team Split for 3 Interns

Role	Main Responsibilities	Coordination Notes
Intern A - Backend/Auth Lead	Backend API, authentication, authorization, invitation flow, database migrations, validation, permission tests.	Must coordinate closely with Intern B for API contracts and Intern C for collision service integration.
Intern B - Frontend/Product Lead	UI layout, forms, dashboards, timetable grid, exam list/calendar views, filtering, export UI, usability fixes.	Must prepare wireframes early and keep UI usable even if drag-and-drop is not completed.
Intern C - Scheduling/QA Lead	Slot model, collision detection engine, test data, unit tests, conflict reports, edge cases, deployment support.	Owns the correctness of conflict detection and prepares the demo scenarios that prove it works.

8.1 Collaboration Rules

- Use an issue board with clear task ownership and acceptance criteria.
- Make small pull requests and integrate at least every 1-2 days.
- Define API contracts before frontend and backend work diverge.
- Keep a project decision log for changed requirements or unresolved quirks.
- End every week with a working demo, even if some features are incomplete.

9. Four-Week Roadmap

The schedule below assumes five working days per week. Adjust the exact days based on holidays, supervisor meetings, or internship logistics, but keep the weekly deliverables fixed.

Week	Theme	Main Activities	Expected Deliverable
Week 1	Foundation and design	Confirm requirements; choose tech stack; create repository; design database; define roles; create wireframes; implement login skeleton; prepare seed data.	Reviewed schema, wireframes, issue board, running empty project, initial auth/database migration.
Week 2	Core MVP data and weekly timetable	Implement workgroup/department/course/classroom CRUD; implement invitation flow; build weekly slot model and timetable grid; add weekly collision checks v1.	Admin can invite users; courses/classrooms can be entered; weekly schedule entries can be created; basic conflicts are detected.

Week 3	Exam scheduling and full collision engine	Implement midterm/final/makeup exam module; complete collision service; build conflict report page; add filters; start exports; write unit tests.	Exam schedules can be created; weekly and exam conflicts are reported clearly; conflict tests pass; basic reports are available.
Week 4	Hardening, deployment, and final demo	Fix bugs; improve UI; complete exports; add validation; prepare README; deploy demo; create final presentation and demo script.	End-to-end demonstrable system, documentation, test accounts, seed data, final demo, and supervisor handover.

9.1 Suggested Weekly Checkpoints

Checkpoint	Review Question
End of Week 1	Can the team explain the data model, permissions, UI flow, and unresolved requirements? Does the app run locally for all interns?
End of Week 2	Can an admin create departments, invite a user, add classrooms/courses, and place a weekly class with collision detection?
End of Week 3	Can users schedule midterm/final/makeup exams and receive meaningful conflict reports? Are unit tests covering collision cases?
End of Week 4	Can a supervisor use the demo without developer help? Are export files, documentation, and demo scenarios ready?

9.2 Day-Level Guidance

Period	Focus
Week 1, Days 1-2	Kickoff, requirements clarification, backlog, role split, tech stack selection.
Week 1, Days 3-5	Database schema, wireframes, initial project skeleton, seed data, basic auth scaffold.
Week 2, Days 1-2	Core CRUD APIs and pages for departments, classrooms, courses, memberships.
Week 2, Days 3-5	Invitation flow, weekly grid, schedule entry forms, weekly collision v1.
Week 3, Days 1-2	Exam data model and exam scheduling UI/API.
Week 3, Days 3-5	Complete collision service, conflict report, export v1, tests.
Week 4, Days 1-2	Bug fixing, validation, usability improvements, permission hardening.
Week 4, Days 3-5	Deployment, documentation, final presentation, demo rehearsal, handover.

10. Acceptance Criteria

10.1 Functional Acceptance Criteria

- Admin can create departments and invite at least two department sub-accounts using university email addresses.
- Invited user can complete account creation through a tokenized invitation link.
- Department sub-account can manage only assigned department course records.
- Admin and permitted users can create classrooms with building, classroom ID, and capacity.
- A weekly course session can be scheduled into valid Monday-Friday 08:30-17:30 slots.
- The system blocks or clearly warns about classroom, lecturer, and cohort conflicts.
- Midterm, final, and makeup exam records can be created, edited, filtered, and exported.
- Exam collision detection works for classroom, lecturer/responsible person, duplicate exam, and cohort conflicts.
- Dashboard and reports make unresolved conflicts visible.
- A non-technical supervisor can run the demo scenario from the README.

10.2 Technical Acceptance Criteria

- Application can be installed and run from the documented setup steps.
- Database schema uses clear relationships and constraints.
- Collision detection is covered by unit tests, including boundary cases where one session ends exactly when another begins.
- Authorization is enforced server-side, not only hidden in the UI.
- Secrets are read from environment variables or configuration files excluded from version control.
- The codebase is organized into clear modules/components and can be extended by a future team.

10.3 Demo Scenarios

1. Admin creates a workgroup, two departments, three classrooms, and invites two department sub-accounts.
2. Sub-account completes account setup and adds a course list for the assigned department.
3. User schedules a weekly class successfully, then tries to schedule another class in the same room and time; the system displays a classroom conflict.
4. User schedules a same-year/same-semester course at the same time; the system displays a cohort conflict or warning.
5. User creates a final exam, then attempts to add another exam in the same classroom at an overlapping time; the system blocks it.
6. Admin opens the conflict report, fixes a conflict, exports the weekly timetable and exam schedule.

11. Initial Backlog and Priority

Priority	Items
Must	Authentication, roles, invitation completion, workgroup/departments, courses, classrooms, weekly schedule, exam schedule, collision detection, basic reports.
Should	Filtering, dashboard counters, CSV/XLSX export, conflict report page, seed data, permission tests, deployment documentation.
Could	Drag-and-drop timetable, PDF export, lecturer entity, multi-room exam split, proctor assignment, classroom equipment filters.
Won't for MVP	Automatic optimizer, LDAP/SSO, official OBS/SIS integration, mobile app, advanced analytics.

11.1 Risks and Mitigation

Risk	Description	Mitigation
Scope creep	New scheduling quirks appear during development.	Record all new rules; implement only MVP-critical rules first; move extras to backlog.
Collision logic bugs	Incorrect conflict detection reduces trust in the system.	Create unit tests with edge cases and demo scenarios before UI polish.
Frontend/backend mismatch	UI depends on endpoints that are not ready.	Define API contracts in Week 1 and use mock data if needed.
Email delivery issues	SMTP setup may fail or be unavailable.	Use mock email/sandbox mode for MVP and document real SMTP configuration.
Authorization gaps	Users may access another department/workgroup data.	Add server-side checks and test unauthorized requests.
Deployment delays	The team spends final days fixing environment issues.	Prepare deployment path by Week 3, not only at the end.

12. Supervisor Notes for Requirement Clarification

- Should department sub-accounts be allowed to delete shared classrooms, or only create/edit their own classroom suggestions?
- Should hard conflicts always block saving, or should admins be able to override with a justification?
- Should cohort conflicts be based on department/year/semester only, or should elective courses be treated differently?
- Are midterms expected to occur during normal weekly schedule weeks, and should exam-vs-course conflicts be checked?
- Is classroom capacity required for courses and exams, and where should expected student count come from?
- Should the lecturer be a free-text field in the MVP or a managed entity?
- Which export format is required for actual use: CSV, XLSX, PDF, or all of them?

These clarification questions should be discussed during the first week. The team should keep decisions in the project log so later implementation choices remain traceable.